Static vs. Dynamic Typing

At the most abstract level, there are two different strategies for dealing with type information in a programing language: *static* or *dynamic*.

A language with static types requires that each variable be assigned a type when it is first declared. The language expects to know what the type of a value is and uses that information to constrain at compile time what values can be assigned to that variable.

Different static languages have different requirements for how types are assigned. Some languages, like Java, require the type of every variable to be explicitly stated when the variable is declared. Other languages, like TypeScript, allow for *type inference*. In TypeScript, if you assign a variable with a typical JavaScript assignment like this:

```
let x = "hello"
```

TypeScript infers from the assignment that `x` is meant to be a string, and does not require further information; you do not have to explicitly declare that `x` is a string. Later, if we try to say `x = 3`, the TypeScript compiler will flag this as an error because `3` is not a string.

Some static languages also infer that if there's a type like `string`, there is a type "array of string," whereas in others you need to explicitly define the existence of the array. Some languages require you to declare up front whether a value can be null; others don't.

On the other hand, a dynamically typed language, like Ruby or plain JavaScript, assigns types at run time. In a dynamic language, you do not need to ever declare the type of a variable in the code. The language checks type information at run time using the current value of a variable at the moment the language needs to determine behavior—for example, when a method is called on a variable.

Types still have meaning in a dynamic language even if the type is not explicitly